

MANUAL TÉCNICO Y DE OPERACIÓN

Objetivos

Objetivo General

Desarrollar una representación tridimensional interactiva inspirada en la "Casa de Clarence", integrando técnicas avanzadas de modelado, texturizado, iluminación y animación mediante el uso de la API OpenGL y el lenguaje C++. Aplicar los conocimientos del curso para desarrollar un entorno interactivo en 3D utilizando C++ y la API de OpenGL.

Específicos:

- Modelado:** Recrear la "Casa de Clarence" integrando modelos externos de Blender y objetos decorativos programados manualmente.
- Apariencia:** Implementar un sistema de iluminación dinámica (luces de punto y foco) y aplicar texturas detalladas a todos los elementos.
- Animación:** Programar movimientos lógicos y orgánicos, como el ondeo de la bandera y el caminar del pájaro, controlados por funciones matemáticas y teclado.

Diagrama de flujo

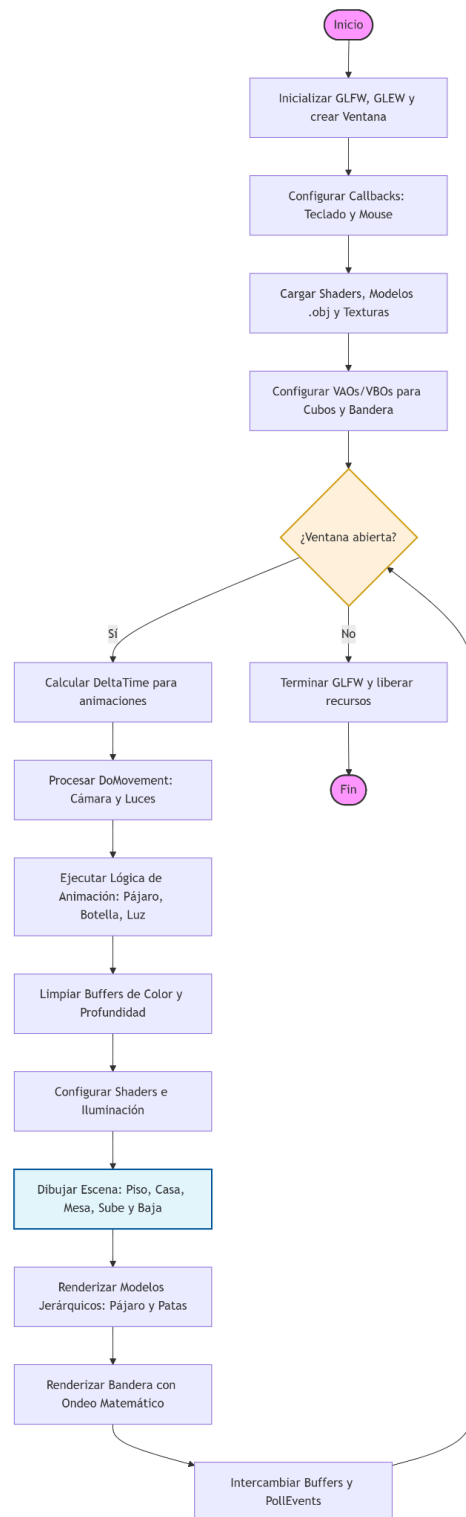
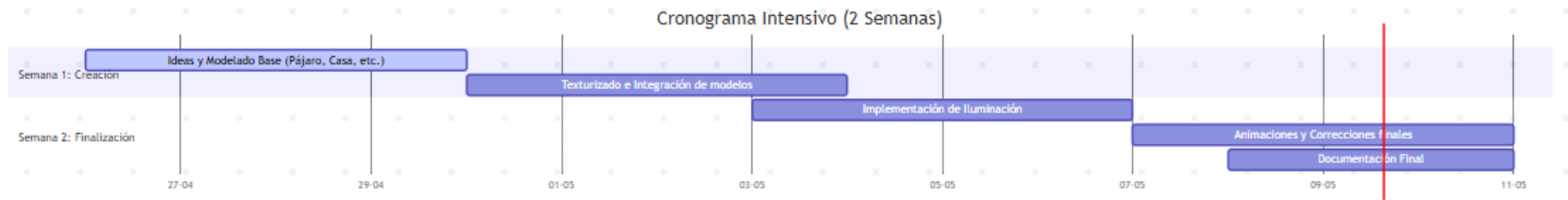


Diagrama de Gantt



-Conceptualización y Modelado (26 - 29 de abril): Tras una hora inicial de lluvia de ideas, se dedicaron los primeros días a la creación de las mallas base en 3D, incluyendo modelos como la casa, el pájaro y el piso.

-Texturizado e Integración (30 de abril - 3 de mayo): Se aplicaron las texturas (archivos .png como el del poste y la botella) y se establecieron las jerarquías de los modelos para unir las piezas correctamente.

-Iluminación de la Escena (3 - 6 de mayo): Se configuraron los Shaders para implementar los distintos tipos de luces: ambiental, direccional, puntuales y el spotlight que sigue a la cámara.

-Animación, Corrección y Documentación (7 - 10 de mayo): Se programó la lógica de movimiento (como el caminar del pájaro y el ondeo de la bandera), se pulieron detalles técnicos y se redactó el informe final.

Alcance:

Modelado e Integración de Activos

La escena se compone de dos tipos de activos geométricos:

Modelos Importados (Blender): Diseño y exportación de mallas complejas que incluyen la estructura principal de la residencia y un modelo de ave con estructura jerárquica (dividido en piezas) para facilitar su articulación.

Modelos Procedurales y Primitivas (OpenGL): Implementación directa en código de objetos decorativos y mobiliario mediante arreglos de vértices (VAOs/VBOs):

- Mobiliario: Silla, mesa y botella.
- Exteriores: Poste de luz, sombrilla con mástil, antena de TV y un juego de sube y baja.

Apariencia Visual y Sombreado

Mapeo de Texturas: Aplicación de texturas envolventes y mapas de bits para aumentar el realismo de los materiales en todos los objetos de la escena.

Sistema de Iluminación: Implementación de distintos tipos de fuentes de luz (Ambiental, Direccional, Puntual y Focal/Spotlight) para generar profundidad y atmósfera en el entorno.

Sistemas de Animación

Se implementaron seis cinemáticas distintas controladas matemáticamente y mediante eventos:

Animaciones Automáticas: Oscilación del sube y baja, movimiento de exploración de la antena de TV, ondeo dinámico de la bandera mediante funciones trigonométricas.

Animaciones por Interacción (Teclado): Ciclo de caminata del pájaro (articulación de extremidades), giro rotacional de la botella sobre su eje, cinemática de caída de la luminaria (foco) con desactivación de fuente de luz al impacto.

Limitantes

Falta de Visualización en Tiempo Real: A diferencia de programas como Blender, en OpenGL no podemos ver lo que estamos construyendo mientras escribimos el código. Esto nos obligó a compilar y ejecutar el programa muchas veces para corregir la posición o el tamaño de cada objeto.

Complejidad en las Texturas: Ajustar las imágenes sobre los objetos fue difícil porque las coordenadas de textura se calculan manualmente. Si una imagen se veía estirada o chueca, no lo sabíamos hasta correr el programa, lo que hizo que el proceso fuera lento.

Cálculos Matemáticos Manuales: Todo el posicionamiento y las animaciones dependen de matrices y funciones matemáticas. Un pequeño error en un número podía hacer que un objeto desapareciera de la pantalla o se moviera de forma extraña, complicando la detección de errores.

Metodología de software aplicada

Para la gestión del proyecto se implementó la metodología de Cascada, elegida por su enfoque lineal y secuencial. Este método permitió establecer un orden riguroso donde el inicio de una etapa dependía estrictamente de la finalización de la anterior, garantizando la integridad técnica de la escena 3D.

Fases de la Implementación:

Fase de Modelado (Geometría): Se priorizó la creación de la malla base (vértices y caras) de todos los objetos en OpenGL y Blender. Como se identificó en la planeación, era imposible avanzar sin tener definida la estructura geométrica.

Fase de Texturizado e Iluminación: Una vez finalizada la geometría, se procedió al mapeo de coordenadas UV para la aplicación de texturas (.png). Paralelamente, se configuraron los Shaders de iluminación, ya que el material y la luz interactúan directamente sobre la malla ya terminada.

Fase de Animación y Dinámica: Con los objetos visualmente terminados, se integró la lógica de movimiento mediante funciones matemáticas y callbacks de teclado. Esta etapa se dejó al final para asegurar que las transformaciones de matriz se aplicaran sobre modelos completos y texturizados.

Se optó por este modelo debido a las dependencias críticas del motor gráfico; por ejemplo, el mapeo de texturas requiere obligatoriamente que el arreglo de vértices (VBO) esté previamente definido, y las animaciones jerárquicas requieren que el modelo esté correctamente integrado en el escenario.

Documentación del código

1. Cabeceras y Librerías

Se integran las API fundamentales para el desarrollo del motor gráfico:

- GLEW/GLFW: Gestión de ventanas y extensiones de OpenGL.
- GLM: Biblioteca matemática para el cálculo de matrices y vectores.
- Carga de Recursos: SOIL2 y stb_image para el procesamiento de texturas.
- Abstracción: Inclusión de clases propias (Shader.h, Camera.h, Model.h) para organizar la carga de objetos .obj y sombreadores.

```
#include <iostream>
```

```
#include <cmath>
```

```
// GLEW y GLFW: Manejo de ventana y extensiones de OpenGL
```

```
#include <GL/glew.h>
```

```
#include <GLFW/glfw3.h>
```

```
// Librerías de Imágenes y Matemáticas
#include "stb_image.h"
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
#include "SOIL2/SOIL2.h"

// Clases Auxiliares (Abstracción de Shaders, Cámara y Modelos)
#include "Shader.h"
#include "Camera.h"
#include "Model.h"
```

2. Variables Globales y Estado

Se declaran los objetos de persistencia que mantienen el estado de la escena, como la configuración de la cámara y las banderas booleanas que actúan como interruptores para el flujo de las animaciones. Asimismo, se integran los arreglos de vértices definidos manualmente para geometrías básicas, permitiendo un control directo sobre los atributos de posición, normales de superficie y coordenadas de texturizado.

```
// Cámara y Control de Tiempo
Camera camera(glm::vec3(0.0f, 0.0f, 3.0f));
GLfloat deltaTime = 0.0f;
GLfloat lastFrame = 0.0f;

// Interruptores de Animación (Booleans)
bool AnimBall = false; // Pájaro
bool luzAnim = false; // Caída de luz
bool animBotella = false; // Rotación botella

// Datos Geométricos (Vértices manuales para Mesa, Poste y Bandera)
float vertices[] = { ... }; // Coordenadas del cubo base
GLfloat flagVertices[] = { ... }; // Coordenadas de los segmentos de la bandera

// Atributos de Iluminación
glm::vec3 pointLightPositions[] = {
    glm::vec3(0.95f, 6.45f, 10.0f), // Luz inicial en el poste
    // ... otras posiciones
};
```

3. Configuración Inicial (main)

La función principal establece la resolución de pantalla y configura la transferencia inicial de datos a la VRAM. Durante esta etapa se instancian los modelos externos y se configuran las funciones de interrupción para periféricos.

```
// Creación de Ventana
```

```
GLFWwindow* window = glfwCreateWindow(WIDTH, HEIGHT, "Fuentes de luz", nullptr, nullptr);
glViewport(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT);

// Instancia de Modelos Externos
Model Casa((char*)"Models/casa2 - copia.obj");
Model Piso((char*)"Models/piso.obj");
Model pajar((char*)"Models/CR.obj");

// Función Lambda para colores sólidos rápidos
auto generarColor = [](GLuint id, GLubyte r, GLubyte g, GLubyte b) {
    glBindTexture(GL_TEXTURE_2D, id);
    GLubyte pixel[] = { r, g, b };
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, 1, 1, 0, GL_RGB, GL_UNSIGNED_BYTE,
pixel);
    glGenerateMipmap(GL_TEXTURE_2D);
};
generarColor(texRojo, 200, 0, 0);
```

4. El Bucle de Renderizado

El ciclo principal sincroniza el movimiento mediante el cálculo del DeltaTime, garantizando que las animaciones sean independientes de la frecuencia de actualización del hardware. En cada iteración, se actualizan los uniformes de las fuentes lumínicas en el sombreador y se recalculan las matrices de vista y proyección necesarias para transformar las coordenadas del mundo 3D en una imagen bidimensional para el observador.

```
while (!glfwWindowShouldClose(window)) {
    // 1. Cálculo de DeltaTime
    GLfloat currentFrame = glfwGetTime();
    deltaTime = currentFrame - lastFrame;
    lastFrame = currentFrame;

    // 2. Definición de Perspectiva (Proyección)
    glm::mat4 projection = glm::perspective(camera.GetZoom(), (GLfloat)SCREEN_WIDTH /
(GLfloat)SCREEN_HEIGHT, 0.1f, 100.0f);

    // 3. Envío de Iluminación al Shader (Point Light 1 ejemplo)
    glUniform3f(glGetUniformLocation(lightningShader.Program, "pointLights[0].position"),
pointLightPositions[0].x, pointLightPositions[0].y, pointLightPositions[0].z);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "pointLights[0].ambient"),
lightColor.x, lightColor.y, lightColor.z);

    // 4. SpotLight (Linterna de la cámara)
    glUniform3f(glGetUniformLocation(lightningShader.Program, "spotLight.position"),
camera.GetPosition().x, camera.GetPosition().y, camera.GetPosition().z);
    glUniform3f(glGetUniformLocation(lightningShader.Program, "spotLight.direction"),
camera.GetFront().x, camera.GetFront().y, camera.GetFront().z);
```

```
}
```

5. Renderizado Jerárquico y Animación

El sistema emplea transformaciones jerárquicas donde una matriz maestra define la posición global y las piezas secundarias heredan este estado mediante multiplicaciones matriciales sucesivas. Para el ondeo de la bandera, se implementa una animación procedimental que utiliza funciones trigonométricas con desfases temporales, simulando algorítmicamente el comportamiento dinámico de una superficie flexible bajo viento.

```
glm::mat4 modelMaster = glm::mat4(1.0f);  
modelMaster = glm::translate(modelMaster, pagPos); // Matriz Padre  
  
// Dibujar Pata 1 (Hereda posición del cuerpo + rotación propia)  
model = modelMaster;  
model = glm::rotate(model, glm::radians(p1), glm::vec3(0.0f, 0.0f, 1.0f));  
pata1.Draw(lightningShader);  
  
// --- Ejemplo Bandera (Ondeo dinámico con for) ---  
for (int i = 0; i < 4; i++) {  
    float angulo = sin(tiempo * velocidad - (i * desfase)) * amplitud;  
    matrizOndeo = glm::rotate(matrizOndeo, glm::radians(angulo), glm::vec3(0, 1, 0));  
    glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(matrizOndeo));  
    glDrawArrays(GL_TRIANGLES, i * 6, 6);  
    matrizOndeo = glm::translate(matrizOndeo, glm::vec3(0.2f, 0.0f, 0.0f));  
}
```

6. Sistema de Eventos (Callbacks)

La interactividad se gestiona mediante el monitoreo de periféricos de entrada. El KeyCallback procesa eventos únicos para cambios de estado, como el encendido de luces, mientras que DoMovement realiza un sondeo continuo para permitir el desplazamiento fluido de la cámara. La orientación espacial se ajusta a través del MouseCallback, que traduce el desplazamiento del cursor en cambios angulares de dirección.

```
// Pulsaciones Únicas (Interruptores)  
void KeyCallback(GLFWwindow* window, int key, int scancode, int action, int mode) {  
    if (keys[GLFW_KEY_N]) AnimBall = !AnimBall; // Activa caminata  
    if (keys[GLFW_KEY_P]) luzAnim = !luzAnim; // Activa caída de foco  
}  
  
// Movimiento Continuo (Cámara)  
void DoMovement() {  
    if (keys[GLFW_KEY_W]) camera.ProcessKeyboard(FORWARD, deltaTime);  
    if (keys[GLFW_KEY_S]) camera.ProcessKeyboard(BACKWARD, deltaTime);  
}
```



```
// Rotación de Cámara (Mouse)
```

```
void MouseCallback(GLFWwindow* window, double xPos, double yPos) {  
    camera.ProcessMouseMovement(xOffset, yOffset);  
}
```

7. Lógica de Animación (Animation())

Esta subrutina define la física del proyecto mediante la actualización de parámetros en función del tiempo. Se calcula la cinemática de las extremidades del modelo mediante oscilaciones angulares controladas dentro de un rango específico. Para la caída del foco, se implementa una reducción constante en el eje vertical que, al detectar la colisión con el plano del suelo, anula el vector de emisión luminosa para simular que la fuente se ha apagado.

```
void Animation() {  
    if (AnimBall) {  
        float velocidadPatas = 100.0f * deltaTime;  
        if (!step) {  
            p1 += velocidadPatas; p2 -= velocidadPatas;  
            if (p1 > 25.0f) step = true;  
        } else {  
            p1 -= velocidadPatas; p2 += velocidadPatas;  
            if (p1 < -25.0f) step = false;  
        }  
        pagPos.x -= 1.3f * deltaTime; // Desplazamiento del cuerpo  
    }  
  
    if (luzAnim) {  
        ml -= 10.0f * deltaTime;  
        pointLightPositions[0].y = 6.45f + ml;  
        if (pointLightPositions[0].y <= 0.0f) { // Colisión con el suelo  
            Light1 = glm::vec3(0.0f); // Apagar luz  
            luzAnim = false;  
        }  
    }  
}
```

Código completo

```
#include <iostream>  
#include <cmath>  
  
// GLEW  
#include <GL/glew.h>  
  
// GLFW  
#include <GLFW/glfw3.h>  
  
// Other Libs
```

```
#include "stb_image.h"

// GLM Mathematics
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>

//Load Models
#include "SOIL2/SOIL2.h"

// Other includes
#include "Shader.h"
#include "Camera.h"
#include "Model.h"

// Function prototypes
void KeyCallback(GLFWwindow* window, int key, int scancode, int action, int mode);
void MouseCallback(GLFWwindow* window, double xPos, double yPos);
void DoMovement();
void Animation(); // Prototipo para la animación

// Window dimensions
const GLuint WIDTH = 1280, HEIGHT = 720;
int SCREEN_WIDTH, SCREEN_HEIGHT;

// Camera
Camera camera(glm::vec3(0.0f, 0.0f, 3.0f));
GLfloat lastX = WIDTH / 2.0;
GLfloat lastY = HEIGHT / 2.0;
bool keys[1024];
bool firstMouse = true;
// Light attributes
glm::vec3 lightPos(0.0f, 0.0f, 0.0f);
bool active;

// Variables de Animación para teclado
bool AnimBall = false;
bool luzAnim = false;
bool animBotella = false;
float p1 = 0.0f;
float p2 = 0.0f;
glm::vec3 pagPos(-10.5f, 5.5f, -15.0f); // Iniciado en la altura de tu mesa (12.0)
bool step = false;
float ml = 0.0f;
float rotBotella = 0.0f;

// Positions of the point lights
glm::vec3 pointLightPositions[] = {
    glm::vec3(0.95f, 6.45f, 10.0f),
    glm::vec3(0.0f, 0.0f, 0.0f),
    glm::vec3(0.0f, 0.0f, 0.0f),
    glm::vec3(0.0f, 0.0f, 0.0f)
};

float vertices[] = {
    // Posiciones (X,Y,Z)    // Normales (Nx,Ny,Nz)    // UVs (U,V)
    -1.0000f, 1.0000f, -1.0000f, -0.0000f, 1.0000f, -0.0000f, 1.0000f, 1.0000f,
    1.0000f, 1.0000f, 1.0000f, -0.0000f, 1.0000f, -0.0000f, 0.0000f, 0.0000f,
```

```
1.0000f, 1.0000f, -1.0000f, -0.0000f, 1.0000f, -0.0000f, 1.0000f, 0.0000f,
1.0000f, 1.0000f, 1.0000f, -0.0000f, -0.0000f, 1.0000f, 1.0000f, 1.0000f,
-1.0000f, -1.0000f, 1.0000f, -0.0000f, -0.0000f, 1.0000f, 0.0000f, 0.0000f,
1.0000f, -1.0000f, 1.0000f, -0.0000f, -0.0000f, 1.0000f, 1.0000f, 0.0000f,

-1.0000f, 1.0000f, 1.0000f, -1.0000f, -0.0000f, -0.0000f, 1.0000f, 1.0000f,
-1.0000f, -1.0000f, -1.0000f, -1.0000f, -0.0000f, -0.0000f, 0.0000f, 0.0000f,
-1.0000f, -1.0000f, 1.0000f, -1.0000f, -0.0000f, -0.0000f, 1.0000f, 0.0000f,
1.0000f, -1.0000f, -1.0000f, -0.0000f, -1.0000f, -0.0000f, 1.0000f, 1.0000f,
-1.0000f, -1.0000f, 1.0000f, -0.0000f, -1.0000f, -0.0000f, 0.0000f, 0.0000f,
-1.0000f, -1.0000f, -1.0000f, -0.0000f, -1.0000f, -0.0000f, 1.0000f, 0.0000f,

1.0000f, 1.0000f, -1.0000f, 1.0000f, -0.0000f, -0.0000f, 1.0000f, 1.0000f,
1.0000f, -1.0000f, 1.0000f, 1.0000f, -0.0000f, -0.0000f, 0.0000f, 0.0000f,
1.0000f, -1.0000f, -1.0000f, 1.0000f, -0.0000f, -0.0000f, 1.0000f, 0.0000f,
-1.0000f, 1.0000f, 1.0000f, -0.0000f, -0.0000f, -1.0000f, 1.0000f, 1.0000f,
1.0000f, -1.0000f, -1.0000f, -0.0000f, -0.0000f, -1.0000f, 0.0000f, 0.0000f,
-1.0000f, -1.0000f, -1.0000f, -0.0000f, -0.0000f, -1.0000f, 1.0000f, 0.0000f,

-1.0000f, 1.0000f, -1.0000f, -0.0000f, 1.0000f, -0.0000f, 1.0000f, 1.0000f,
-1.0000f, 1.0000f, 1.0000f, -0.0000f, 1.0000f, -0.0000f, 0.0000f, 1.0000f,
1.0000f, 1.0000f, 1.0000f, -0.0000f, 1.0000f, -0.0000f, 0.0000f, 0.0000f,
1.0000f, 1.0000f, 1.0000f, -0.0000f, -0.0000f, 1.0000f, 1.0000f, 1.0000f,
-1.0000f, 1.0000f, 1.0000f, -0.0000f, -0.0000f, 1.0000f, 0.0000f, 1.0000f,
-1.0000f, -1.0000f, 1.0000f, -0.0000f, -0.0000f, 1.0000f, 0.0000f, 0.0000f,

-1.0000f, 1.0000f, 1.0000f, -1.0000f, -0.0000f, -0.0000f, 1.0000f, 1.0000f,
-1.0000f, 1.0000f, -1.0000f, -1.0000f, -0.0000f, -0.0000f, 0.0000f, 1.0000f,
-1.0000f, -1.0000f, -1.0000f, -1.0000f, -0.0000f, -0.0000f, 0.0000f, 0.0000f,
1.0000f, -1.0000f, -1.0000f, -0.0000f, -1.0000f, -0.0000f, 1.0000f, 1.0000f,
1.0000f, -1.0000f, 1.0000f, -0.0000f, -1.0000f, -0.0000f, 0.0000f, 1.0000f,
-1.0000f, -1.0000f, 1.0000f, -0.0000f, -1.0000f, -0.0000f, 0.0000f, 0.0000f,

1.0000f, 1.0000f, -1.0000f, 1.0000f, -0.0000f, -0.0000f, 1.0000f, 1.0000f,
1.0000f, 1.0000f, 1.0000f, 1.0000f, -0.0000f, -0.0000f, 0.0000f, 1.0000f,
1.0000f, -1.0000f, 1.0000f, 1.0000f, -0.0000f, -0.0000f, 0.0000f, 0.0000f,
-1.0000f, 1.0000f, -1.0000f, -0.0000f, -0.0000f, -1.0000f, 1.0000f, 1.0000f,
1.0000f, 1.0000f, -1.0000f, -0.0000f, -0.0000f, -1.0000f, 0.0000f, 1.0000f,
1.0000f, -1.0000f, -1.0000f, -0.0000f, -0.0000f, -1.0000f, 0.0000f, 0.0000f,
};

// --- BANDERA ---
GLfloat flagVertices[] = {
    // Seg 1 (Base): Alto 0.6 -> 0.45
    0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, 1.0f, 0.2f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.25f, 1.0f, 0.0f, -0.6f, 0.0f,
    0.0f, 0.0f, 1.0f, 0.0f, 0.0f,
    0.0f, -0.6f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f, 0.2f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.25f, 1.0f, 0.2f, -0.45f, 0.0f,
    0.0f, 0.0f, 1.0f, 0.25f, 0.25f,

    // Seg 2: Alto 0.45 -> 0.3
    0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.25f, 1.0f, 0.2f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.5f, 1.0f, 0.0f, -0.45f, 0.0f,
    0.0f, 0.0f, 1.0f, 0.25f, 0.25f,
    0.0f, -0.45f, 0.0f, 0.0f, 0.0f, 1.0f, 0.25f, 0.25f, 0.2f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.5f, 1.0f, 0.2f, -0.3f, 0.0f,
    0.0f, 0.0f, 1.0f, 0.5f, 0.5f,

    // Seg 3: Alto 0.3 -> 0.15
    0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.5f, 1.0f, 0.2f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.75f, 1.0f, 0.0f, -0.3f, 0.0f,
    0.0f, 0.0f, 1.0f, 0.5f, 0.5f,
    0.0f, -0.3f, 0.0f, 0.0f, 0.0f, 1.0f, 0.5f, 0.5f, 0.2f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.75f, 1.0f, 0.2f, -0.15f, 0.0f,
    0.0f, 0.0f, 1.0f, 0.75f, 0.75f,
```

```
// Seg 4 (Punta): Alto 0.15 -> 0
0.0f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.75f, 1.0f, 0.2f, -0.075f, 0.0f, 0.0f, 0.0f, 1.0f, 1.0f, 0.5f, 0.0f, -0.15f, 0.0f,
0.0f, 0.0f, 1.0f, 0.75f, 0.0f
};
glm::vec3 Light1 = glm::vec3(0);

// Deltatime
GLfloat deltaTime = 0.0f; // Time between current frame and last frame
GLfloat lastFrame = 0.0f; // Time of last frame

int main()
{
    // Init GLFW
    glfwInit();
    // Set all the required options for GLFW
    /*glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
    glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
    glfwWindowHint(GLFW_RESIZABLE, GL_FALSE);*/

    // Create a GLFWwindow object that we can use for GLFW's functions
    GLFWwindow* window = glfwCreateWindow(WIDTH, HEIGHT, "Fuentes de luz", nullptr, nullptr);

    if (nullptr == window)
    {
        std::cout << "Failed to create GLFW window" << std::endl;
        glfwTerminate();

        return EXIT_FAILURE;
    }

    glfwMakeContextCurrent(window);

    glfwGetFramebufferSize(window, &SCREEN_WIDTH, &SCREEN_HEIGHT);

    // Set the required callback functions
    glfwSetKeyCallback(window, KeyCallback);
    glfwSetCursorPosCallback(window, MouseCallback);

    // GLFW Options
    glfwSetInputMode(window, GLFW_CURSOR, GLFW_CURSOR_DISABLED);

    // Set this to true so GLEW knows to use a modern approach to retrieving function pointers and
    extensions
    glewExperimental = GL_TRUE;
    // Initialize GLEW to setup the OpenGL Function pointers
    if (GLEW_OK != glewInit())
    {
        std::cout << "Failed to initialize GLEW" << std::endl;
        return EXIT_FAILURE;
    }

    // Define the viewport dimensions
    glViewport(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT);

    Shader shader("Shader/modelLoading.vs", "Shader/modelLoading.frag");
    Shader lightingShader("Shader/lighting.vs", "Shader/lighting.frag");
```

```
Shader lampShader("Shader/lamp.vs", "Shader/lamp.frag");

Model Casa((char*)"Models/casa2 - copia.obj");
Model Piso((char*)"Models/piso.obj");
Model pata1((char*)"Models/PR1.obj");
Model pata2((char*)"Models/PR2obj.obj");
Model pajaro((char*)"Models/CR.obj");

// CUBO NORMAL
GLuint VBO, VAO;
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);
glBindVertexArray(VAO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

// Position attribute (X, Y, Z) - Location 0
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(GLfloat), (GLvoid*)0);
glEnableVertexAttribArray(0);

// Normal attribute (Nx, Ny, Nz) - Location 1
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(3 * sizeof(float)));
glEnableVertexAttribArray(1);

// Texture attribute (U, V) - Location 2
glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, 8 * sizeof(GLfloat), (GLvoid*)(6 *
sizeof(GLfloat)));
glEnableVertexAttribArray(2);

// Set texture units
lightingShader.Use();
glUniform1i(glGetUniformLocation(lightingShader.Program, "Material.diffuse"), 0);
glUniform1i(glGetUniformLocation(lightingShader.Program, "Material.specular"), 1);

glm::mat4 projection = glm::perspective(camera.GetZoom(), (GLfloat)SCREEN_WIDTH /
(GLfloat)SCREEN_HEIGHT, 0.1f, 100.0f);

// --- VAO y VBO para la Bandera ---
GLuint flagVAO, flagVBO;
glGenVertexArrays(1, &flagVAO);
glGenBuffers(1, &flagVBO);

glBindVertexArray(flagVAO);
glBindBuffer(GL_ARRAY_BUFFER, flagVBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(flagVertices), flagVertices, GL_STATIC_DRAW);

// Atributo 0 (Posición)
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(GLfloat), (GLvoid*)0);
glEnableVertexAttribArray(0);
// Atributo 1 (Normales)
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(GLfloat), (GLvoid*)(3 *
sizeof(GLfloat)));
glEnableVertexAttribArray(1);
// Atributo 2 (Texturas)
glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, 8 * sizeof(GLfloat), (GLvoid*)(6 *
sizeof(GLfloat)));
glEnableVertexAttribArray(2);

glBindVertexArray(0); // Desvinculamos
```

```
GLuint texture;
glGenTextures(1, &texture);
glBindTexture(GL_TEXTURE_2D, texture);
int textureWidth, textureHeight, nrChannels;
stbi_set_flip_vertically_on_load(true);
unsigned char* image;
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR_MIPMAP_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_NEAREST_MIPMAP_NEAREST);

image = stbi_load("images/poste.png", &textureWidth, &textureHeight, &nrChannels, 0);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, textureWidth, textureHeight, 0, GL_RGB,
GL_UNSIGNED_BYTE, image);
glGenerateMipmap(GL_TEXTURE_2D);
if (image)
{
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, textureWidth, textureHeight, 0, GL_RGB,
GL_UNSIGNED_BYTE, image);
    glGenerateMipmap(GL_TEXTURE_2D);
}
else
{
    std::cout << "Failed to load texture" << std::endl;
}
stbi_image_free(image);

GLuint texture1;
glGenTextures(1, &texture1);
glBindTexture(GL_TEXTURE_2D, texture1);
int textureWidth1, textureHeight1, nrChannels1;
stbi_set_flip_vertically_on_load(true);
unsigned char* blanco;
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR_MIPMAP_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_NEAREST_MIPMAP_NEAREST);

blanco = stbi_load("images/silla.png", &textureWidth1, &textureHeight1, &nrChannels1, 0);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, textureWidth1, textureHeight1, 0, GL_RGB,
GL_UNSIGNED_BYTE, blanco);
glGenerateMipmap(GL_TEXTURE_2D);
if (blanco)
{
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, textureWidth1, textureHeight1, 0,
GL_RGB, GL_UNSIGNED_BYTE, blanco);
    glGenerateMipmap(GL_TEXTURE_2D);
}
else
{
    std::cout << "Failed to load texture" << std::endl;
}
stbi_image_free(blanco);

GLuint texture2;
```

```
glGenTextures(1, &texture2);
glBindTexture(GL_TEXTURE_2D, texture2);
int textureWidth2, textureHeight2, nrChannels2;
stbi_set_flip_vertically_on_load(true);
unsigned char* botella;
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR_MIPMAP_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
GL_NEAREST_MIPMAP_NEAREST);

botella = stbi_load("images/botella.png", &textureWidth2, &textureHeight2, &nrChannels2, 0);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, textureWidth2, textureHeight2, 0, GL_RGB,
GL_UNSIGNED_BYTE, botella);
glGenerateMipmap(GL_TEXTURE_2D);
if (botella)
{
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, textureWidth2, textureHeight2, 0,
GL_RGB, GL_UNSIGNED_BYTE, botella);
    glGenerateMipmap(GL_TEXTURE_2D);
}
else
{
    std::cout << "Failed to load texture" << std::endl;
}
stbi_image_free(botella);

// --- CREACIÓN DE TEXTURAS DE COLOR SÓLIDO (Sin archivos externos) ---
GLuint texRojo, texAmarillo, texAzul, texbeige, texceleste, texGris, texNaranja;
glGenTextures(1, &texRojo);
glGenTextures(1, &texAmarillo);
glGenTextures(1, &texAzul);
glGenTextures(1, &texbeige);
glGenTextures(1, &texceleste);
glGenTextures(1, &texGris);
glGenTextures(1, &texNaranja);

auto generarColor = [](GLuint id, GLubyte r, GLubyte g, GLubyte b) {
    glBindTexture(GL_TEXTURE_2D, id);
    GLubyte pixel[] = { r, g, b }; // Definimos el color RGB
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, 1, 1, 0, GL_RGB,
GL_UNSIGNED_BYTE, pixel);
    glGenerateMipmap(GL_TEXTURE_2D);
};

generarColor(texRojo, 200, 0, 0);
generarColor(texAmarillo, 255, 255, 0);
generarColor(texAzul, 0, 100, 200);
generarColor(texbeige, 237, 185, 151);
generarColor(texceleste, 81, 209, 246);
generarColor(texGris, 100, 100, 100);
generarColor(texNaranja, 255, 128, 0);
```

```
//////////////////////////////////////

// Game loop
while (!glfwWindowShouldClose(window))
{

    // Calculate deltatime of current frame
    GLfloat currentFrame = glfwGetTime();
    deltaTime = currentFrame - lastFrame;
    lastFrame = currentFrame;

    // Check if any events have been activiated (key pressed, mouse moved etc.) and call
corresponding response functions
    glfwPollEvents();
    DoMovement();
    Animation(); // Ejecutar lógica de animación

    // Clear the colorbuffer
    glClearColor(0.53f, 0.81f, 0.92f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // OpenGL options
    glEnable(GL_DEPTH_TEST);

    glm::mat4 modelTemp = glm::mat4(1.0f);

    //Load Model

    // Use cooresponding shader when setting uniforms/drawing objects
    lightingShader.Use();

    glUniform1i(glGetUniformLocation(lightingShader.Program, "diffuse"), 0);
    glUniform1i(glGetUniformLocation(lightingShader.Program, "specular"), 1);

    GLint viewPosLoc = glGetUniformLocation(lightingShader.Program, "viewPos");
    glUniform3f(viewPosLoc, camera.GetPosition().x, camera.GetPosition().y,
camera.GetPosition().z);

    // Directional light
    glUniform3f(glGetUniformLocation(lightingShader.Program, "dirLight.direction"), -0.2f,
-1.0f, -0.3f);
    glUniform3f(glGetUniformLocation(lightingShader.Program, "dirLight.ambient"), 0.75f,
0.75f, 0.65f);
    glUniform3f(glGetUniformLocation(lightingShader.Program, "dirLight.diffuse"), 0.5f, 0.5f,
0.5f);
    glUniform3f(glGetUniformLocation(lightingShader.Program, "dirLight.specular"), 0.0f, 0.0f,
0.0f);

    // Point light 1
    glm::vec3 lightColor;
    lightColor.x = abs(sin(glfwGetTime() * Light1.x));
    lightColor.y = abs(sin(glfwGetTime() * Light1.y));
    lightColor.z = sin(glfwGetTime() * Light1.z);

    glUniform3f(glGetUniformLocation(lightingShader.Program, "pointLights[0].position"),
pointLightPositions[0].x, pointLightPositions[0].y, pointLightPositions[0].z);
```



```

        glUniform3f(glGetUniformLocation(lightningShader.Program,
lightColor.x, lightColor.y, lightColor.z);
        glUniform3f(glGetUniformLocation(lightningShader.Program,
lightColor.x, lightColor.y, lightColor.z);
        glUniform3f(glGetUniformLocation(lightningShader.Program, "pointLights[0].specular"), 1.0f,
1.0f, 0.0f);
        glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[0].constant"),
1.0f);
        glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[0].linear"),
0.045f);
        glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[0].quadratic"),
0.075f);

        // Point light 2
        glUniform3f(glGetUniformLocation(lightningShader.Program, "pointLights[1].position"),
pointLightPositions[1].x, pointLightPositions[1].y, pointLightPositions[1].z);
        glUniform3f(glGetUniformLocation(lightningShader.Program, "pointLights[1].ambient"),
0.05f, 0.05f, 0.05f);
        glUniform3f(glGetUniformLocation(lightningShader.Program, "pointLights[1].diffuse"), 0.0f,
0.0f, 0.0f);
        glUniform3f(glGetUniformLocation(lightningShader.Program, "pointLights[1].specular"), 0.0f,
0.0f, 0.0f);
        glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[1].constant"),
1.0f);
        glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[1].linear"), 0.0f);
        glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[1].quadratic"),
0.0f);

        // Point light 3
        glUniform3f(glGetUniformLocation(lightningShader.Program, "pointLights[2].position"),
pointLightPositions[2].x, pointLightPositions[2].y, pointLightPositions[2].z);
        glUniform3f(glGetUniformLocation(lightningShader.Program, "pointLights[2].ambient"), 0.0f,
0.0f, 0.0f);
        glUniform3f(glGetUniformLocation(lightningShader.Program, "pointLights[2].diffuse"), 0.0f,
0.0f, 0.0f);
        glUniform3f(glGetUniformLocation(lightningShader.Program, "pointLights[2].specular"), 0.0f,
0.0f, 0.0f);
        glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[2].constant"),
1.0f);
        glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[2].linear"), 0.0f);
        glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[2].quadratic"),
0.0f);

        // Point light 4
        glUniform3f(glGetUniformLocation(lightningShader.Program, "pointLights[3].position"),
pointLightPositions[3].x, pointLightPositions[3].y, pointLightPositions[3].z);
        glUniform3f(glGetUniformLocation(lightningShader.Program, "pointLights[3].ambient"), 0.0f,
0.0f, 0.0f);
        glUniform3f(glGetUniformLocation(lightningShader.Program, "pointLights[3].diffuse"), 0.0f,
0.0f, 0.0f);
        glUniform3f(glGetUniformLocation(lightningShader.Program, "pointLights[3].specular"), 0.0f,
0.0f, 0.0f);
        glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[3].constant"),
1.0f);
        glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[3].linear"), 0.0f);
        glUniform1f(glGetUniformLocation(lightningShader.Program, "pointLights[3].quadratic"),
0.0f);

        // SpotLight Tipo Lampara depende mucho de la tarjeta grafica,

```

```
glUniform3f(glGetUniformLocation(lightningShader.Program, "spotLight.position"),
camera.GetPosition().x, camera.GetPosition().y, camera.GetPosition().z);
glUniform3f(glGetUniformLocation(lightningShader.Program, "spotLight.direction"),
camera.GetFront().x, camera.GetFront().y, camera.GetFront().z);
glUniform3f(glGetUniformLocation(lightningShader.Program, "spotLight.ambient"), 0.6f, 0.6f,
0.6f);
glUniform3f(glGetUniformLocation(lightningShader.Program, "spotLight.diffuse"), 0.6f, 0.6f,
0.6f);
glUniform3f(glGetUniformLocation(lightningShader.Program, "spotLight.specular"), 0.0f,
0.0f, 0.0f);
glUniform1f(glGetUniformLocation(lightningShader.Program, "spotLight.constant"), 1.0f);
glUniform1f(glGetUniformLocation(lightningShader.Program, "spotLight.linear"), 0.3f);
glUniform1f(glGetUniformLocation(lightningShader.Program, "spotLight.quadratic"), 0.7f);
glUniform1f(glGetUniformLocation(lightningShader.Program, "spotLight.cutOff"),
glm::cos(glm::radians(12.0f)));
glUniform1f(glGetUniformLocation(lightningShader.Program, "spotLight.outerCutOff"),
glm::cos(glm::radians(18.0f)));

// Set material properties
glUniform1f(glGetUniformLocation(lightningShader.Program, "material.shininess"), 16.0f);

// Create camera transformations
glm::mat4 view;
view = camera.GetViewMatrix();

// Get the uniform locations
GLint modelLoc = glGetUniformLocation(lightningShader.Program, "model");
GLint viewLoc = glGetUniformLocation(lightningShader.Program, "view");
GLint projLoc = glGetUniformLocation(lightningShader.Program, "projection");

// Pass the matrices to the shader
glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
glUniformMatrix4fv(projLoc, 1, GL_FALSE, glm::value_ptr(projection));

glm::mat4 model(1);

//Carga de modelo
view = camera.GetViewMatrix();
model = glm::mat4(1);
model = glm::scale(model, glm::vec3(7.11f, 0.0f, 8.1f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
Piso.Draw(lightningShader);
model = glm::mat4(1);

model = glm::mat4(1);
model = glm::translate(model, glm::vec3(-10.0f, 0.2f, -5.0f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
Casa.Draw(lightningShader);

glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, texture);
glUniform1i(glGetUniformLocation(lightningShader.Program, "texture_diffuse"), 0);
glBindVertexArray(VAO);

// =====
// PAJARO
// =====
```

```
// escala, orientación y posición GLOBAL
glm::mat4 modelMaster = glm::mat4(1.0f);
modelMaster = glm::scale(modelMaster, glm::vec3(0.11f));
modelMaster = glm::rotate(modelMaster, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
modelMaster = glm::rotate(modelMaster, glm::radians(-20.0f), glm::vec3(0.5f, 0.0f, 0.0f));
modelMaster = glm::translate(modelMaster, pagPos);

// 2. DIBUJAR CUERPO
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(modelMaster));
pajaro.Draw(lightingShader);

// 3. DIBUJAR PATA 1
model = modelMaster;
model = glm::rotate(model, glm::radians(p1), glm::vec3(0.0f, 0.0f, 1.0f)); // Movimiento de
caminado

glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
pata1.Draw(lightingShader);

// 4. DIBUJAR PATA 2
model = modelMaster; // Copiamos la configuración global
model = glm::rotate(model, glm::radians(p2), glm::vec3(0.0f, 0.0f, 1.0f)); // Movimiento de
caminado

glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
pata2.Draw(lightingShader);

glBindVertexArray(0);
// =====
// DIBUJO DEL POSTE
// =====
glBindVertexArray(VAO);
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, texture);

// --- 1. BASE DEL POSTE
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 3.55f, 10.0f)); // Posición base
model = glm::scale(model, glm::vec3(0.11f, 3.5f, 0.11f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// --- 2. BRAZO HORIZONTAL ---
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.6f, 6.5f, 10.0f));
model = glm::scale(model, glm::vec3(0.55f, 0.11f, 0.11f));
model = glm::rotate(model, glm::radians(10.0f), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, texture1);
glUniform1i(glGetUniformLocation(lightingShader.Program, "texture_diffuse"), 0);
glBindVertexArray(VAO);

lightingShader.Use();
glUniform1i(glGetUniformLocation(lightingShader.Program, "material.diffuse"), 0);
glActiveTexture(GL_TEXTURE0);
```

```
glBindVertexArray(VAO);
```

```
// Definimos una variable para mover toda la silla en conjunto
// Posición de la silla para que esté a un lado del poste
float sillaX = 2.0f;
float sillaZ = 0.1f;
float sillaY = 0.8f;
// =====
// --- 1. ASIENTO ---
// =====
model = glm::mat4(1.0f);
// Elevamos el asiento a Y=0.5 para que las patas queden debajo
model = glm::translate(model, glm::vec3(sillaX, sillaY, sillaZ));
model = glm::scale(model, glm::vec3(0.4f, 0.03f, 0.4f)); // Proporcional al poste
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// =====
// --- 2. RESPALDO ---
// =====
model = glm::mat4(1.0f);
// Posición Y: Centro del respaldo está arriba del asiento
// Posición Z: -0.35 para que esté justo en el borde de atrás del asiento
model = glm::translate(model, glm::vec3(sillaX, sillaY + 0.35f, sillaZ - 0.35f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.03f)); // Delgado y de ancho igual al asiento
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// =====
// --- 3. PATAS ---
// =====
glm::vec3 escalaPata = glm::vec3(0.04f, 0.4f, 0.04f); // Delgadas como el poste
float alturaPata = 0.01f; // Centro de la pata (0.5 de alto / 2)

// Pata Delantera Izquierda
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(sillaX - 0.3f, sillaY - 0.4f, sillaZ + 0.3f));
model = glm::scale(model, escalaPata);
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// Pata Delantera Derecha
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(sillaX + 0.3f, sillaY - 0.4f, sillaZ + 0.3f));
model = glm::scale(model, escalaPata);
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// Pata Trasera Izquierda
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(sillaX - 0.3f, sillaY - 0.4f, sillaZ - 0.3f));
model = glm::scale(model, escalaPata);
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// Pata Trasera Derecha
model = glm::mat4(1.0f);
```

```
model = glm::translate(model, glm::vec3(sillaX + 0.3f, sillaY - 0.4f, sillaZ - 0.3f));
model = glm::scale(model, escalaPata);
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);
```

```
// -----
//          MESA
// -----
```

```
// TABLERO
model = glm::mat4(1);
glBindTexture(GL_TEXTURE_2D, texbeige);
model = glm::scale(model, glm::vec3(1.2f, 0.1f, 0.6));
model = glm::translate(model, glm::vec3(0, 10.0f, 0));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);
```

```
// PATAS
float posiciones[4][3] = {

    { 10.0f, 1.0f, 5.0f}, // Frontal Derecha
    {-10.0f, 1.0f, 5.0f}, // Frontal Izquierda
    { 10.0f, 1.0f, -5.0f}, // Trasera Derecha
    {-10.0f, 1.0f, -5.0f}
```

```
};
;
for (int i = 0; i < 4; i++)
{
```

```
    model = glm::mat4(1);

    model = glm::scale(model, glm::vec3(0.1f, 0.55f, 0.1f));

    model = glm::translate(model, glm::vec3(

        posiciones[i][0],

        posiciones[i][1],

        posiciones[i][2]

    ));

    glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));

    glDrawArrays(GL_TRIANGLES, 0, 36);
```

```
}
```

```
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, texture2);
glUniform1i(glGetUniformLocation(lightningShader.Program, "texture_diffuse"), 0);
glBindVertexArray(VAO);
```

```
// =====
// BOTELLA
```

```
// =====  
glActiveTexture(GL_TEXTURE0);  
glBindTexture(GL_TEXTURE_2D, texture2);  
glUniform1i(glGetUniformLocation(lightingShader.Program, "texture_diffuse"), 0);  
glBindVertexArray(VAO);  
  
float bX = -0.8f;  
float bZ = 0.35f;  
float bY = 1.25f;  
  
model = glm::mat4(1.0f);  
model = glm::translate(model, glm::vec3(bX, bY, bZ));  
model = glm::rotate(model, glm::radians(rotBotella), glm::vec3(0.0f, 1.0f, 0.0f));  
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f));  
  
glm::mat4 botellaBase = model;  
  
// --- CUERPO ---  
model = glm::scale(botellaBase, glm::vec3(0.15f, 0.4f, 0.15f));  
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));  
glDrawArrays(GL_TRIANGLES, 0, 36);  
  
// --- HOMBROS ---  
model = glm::translate(botellaBase, glm::vec3(0.0f, 0.45f, 0.0f));  
model = glm::scale(model, glm::vec3(0.12f, 0.05f, 0.12f));  
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));  
glDrawArrays(GL_TRIANGLES, 0, 36);  
  
// --- CUELLO ---  
model = glm::translate(botellaBase, glm::vec3(0.0f, 0.65f, 0.0f));  
model = glm::scale(model, glm::vec3(0.05f, 0.15f, 0.05f));  
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));  
glDrawArrays(GL_TRIANGLES, 0, 36);  
  
// --- TAPA ---  
model = glm::translate(botellaBase, glm::vec3(0.0f, 0.82f, 0.0f));  
model = glm::scale(model, glm::vec3(0.06f, 0.04f, 0.06f));  
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));  
glDrawArrays(GL_TRIANGLES, 0, 36);  
  
// =====  
// LÁMPARA  
// =====  
lampShader.Use();  
  
GLuint lampModelLoc = glGetUniformLocation(lampShader.Program, "model");  
GLuint lampViewLoc = glGetUniformLocation(lampShader.Program, "view");  
GLuint lampProjLoc = glGetUniformLocation(lampShader.Program, "projection");  
  
glUniformMatrix4fv(lampViewLoc, 1, GL_FALSE, glm::value_ptr(view));  
glUniformMatrix4fv(lampProjLoc, 1, GL_FALSE, glm::value_ptr(projection));  
  
model = glm::mat4(1.0f);  
model = glm::translate(model, pointLightPositions[0]);  
model = glm::scale(model, glm::vec3(0.1f));  
  
glUniformMatrix4fv(lampModelLoc, 1, GL_FALSE, glm::value_ptr(model));  
  
glBindVertexArray(VAO);
```

```
glDrawArrays(GL_TRIANGLES, 0, 36);

lightingShader.Use();

modelLoc = glGetUniformLocation(lightingShader.Program, "model");
viewLoc = glGetUniformLocation(lightingShader.Program, "view");
projLoc = glGetUniformLocation(lightingShader.Program, "projection");

glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
glUniformMatrix4fv(projLoc, 1, GL_FALSE, glm::value_ptr(projection));

glBindVertexArray(VAO);

// =====
// SUBE Y BAJA
// =====
//
// Posición del sube y baja en los 3 ejes
glm::vec3 posSubeBaja = glm::vec3(5.0f, 0.45f, -10.0f);
// ángulo de oscilación (1.5f es la velocidad, 15.0f es la inclinación máxima)
float anguloSB = sin(glfwGetTime() * 1.5f) * 15.0f;

lightingShader.Use();
modelLoc = glGetUniformLocation(lightingShader.Program, "model");
glBindVertexArray(VAO);

// --- MATRIZ PADRE
glm::mat4 matrizSB = glm::translate(glm::mat4(1.0f), posSubeBaja);
matrizSB = glm::rotate(matrizSB, glm::radians(45.0f), glm::vec3(0, 1, 0));

// --- 1. BASE ---
glBindTexture(GL_TEXTURE_2D, texAmarillo);
model = matrizSB;
model = glm::translate(model, glm::vec3(0.0f, 0.1f, 0.0f));
model = glm::scale(model, glm::vec3(0.1f, 0.55f, 0.1f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// --- PARTES MÓVILES ---
// Rotamos alrededor del eje Z local para que suba y baje
glm::mat4 matrizMovil = glm::rotate(matrizSB, glm::radians(anguloSB), glm::vec3(0, 0, 1));

// --- 2. TABLA ---
glBindTexture(GL_TEXTURE_2D, texAzul);
model = matrizMovil;
model = glm::translate(model, glm::vec3(0.0f, 0.52f, 0.0f));
model = glm::scale(model, glm::vec3(2.5f, 0.03f, 0.4f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// --- 3. ASIENTOS---
glBindTexture(GL_TEXTURE_2D, texRojo);
// Asiento Izquierdo
model = matrizMovil;
model = glm::translate(model, glm::vec3(-1.15f, 0.58f, 0.0f));
model = glm::scale(model, glm::vec3(0.2f, 0.05f, 0.3f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
```

```
glDrawArrays(GL_TRIANGLES, 0, 36);

// Asiento Derecho
model = matrizMovil;
model = glm::translate(model, glm::vec3(1.15f, 0.58f, 0.0f));
model = glm::scale(model, glm::vec3(0.2f, 0.05f, 0.3f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

//// =====
////  antena
//// =====

glm::vec3 posAntena = glm::vec3(-11.0f, 6.8f, -12.0f);

float anguloHorizontal = sin glfwGetTime() * 0.5f) * 40.0f;

float factorExtremo = pow(sin glfwGetTime() * 0.5f), 32.0f);

float anguloCabeceo = factorExtremo * sin glfwGetTime() * 1.5f) * 10.0f;
glm::mat4 matrizAntena = glm::translate(glm::mat4(1.0f), posAntena);
matrizAntena = glm::scale(matrizAntena, glm::vec3(1.15f));

// --- 1. SOPORTE BASE y 2. MÁSTIL
glBindTexture(GL_TEXTURE_2D, texture);
model = matrizAntena;
model = glm::translate(model, glm::vec3(0.0f, 0.025f, 0.0f)); // Localmente sobre el techo
model = glm::scale(model, glm::vec3(0.1f, 0.05f, 0.1f)); // Muy aplanado
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// --- 2. MÁSTIL VERTICAL ---
model = matrizAntena;
model = glm::translate(model, glm::vec3(0.0f, 0.15f, 0.0f)); // Elevado
model = glm::scale(model, glm::vec3(0.03f, 0.4f, 0.03f)); // Delgado y alto
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// --- MATRIZ GIRO HORIZONTAL (Afecta al brazo y al plato) ---
glm::mat4 matrizGiroH = glm::rotate(matrizAntena, glm::radians(anguloHorizontal),
glm::vec3(0, 1, 0));

// --- MATRIZ DE PLATO
// Esta matriz hereda el giro horizontal y le suma el movimiento en el eje X local
glm::mat4 matrizPlato = glm::rotate(matrizGiroH, glm::radians(anguloCabeceo), glm::vec3(1,
0, 0));

// --- 3. PLATO ---
glBindTexture(GL_TEXTURE_2D, texceleste);
model = matrizPlato;
model = glm::translate(model, glm::vec3(0.0f, 0.3f, 0.0f));
model = glm::rotate(model, glm::radians(45.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.02f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);
```


del brazo

```
// --- 4. BRAZO INCLINADO (Sale del mástil y rota horizontalmente) ---
glBindTexture(GL_TEXTURE_2D, texture);
model = matrizPlato;
model = glm::translate(model, glm::vec3(0.0f, 0.35f, 0.0f)); // Punto de unión con el mástil
model = glm::rotate(model, glm::radians(45.0f), glm::vec3(1.0f, 0.0f, 0.0f)); // Inclinación fija

model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.15f)); // Extensión del brazo
model = glm::scale(model, glm::vec3(0.01f, 0.01f, 0.5f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// --- 5. LNB
glBindTexture(GL_TEXTURE_2D, texcelestes);
model = matrizPlato;
model = glm::translate(model, glm::vec3(0.0f, 0.95f, -0.6f));
model = glm::rotate(model, glm::radians(45.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.5f));
model = glm::scale(model, glm::vec3(0.04f, 0.04f, 0.04f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// =====s=====
// --- SOMBRILLA ---
// =====

// Posición de la sombrilla
glm::vec3 posSombrilla = glm::vec3(3.0f, 0.0f, -0.8f);

lightingShader.Use();
glUniform1i(glGetUniformLocation(lightingShader.Program, "material.diffuse"), 0);
glActiveTexture(GL_TEXTURE0);
glBindVertexArray(VAO);

// --- MATRIZ PADRE ---
glm::mat4 matrizSombrilla = glm::translate(glm::mat4(1.0f), posSombrilla);
matrizSombrilla = glm::scale(matrizSombrilla, glm::vec3(1.2f, 1.2f, 1.2f));
// --- 1. BASE Y POSTE (Gris - RGB 100,100,100) ---
glBindTexture(GL_TEXTURE_2D, texGris);
// Base
model = glm::translate(matrizSombrilla, glm::vec3(0.0f, 0.05f, 0.0f));
model = glm::scale(model, glm::vec3(0.4f, 0.05f, 0.4f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// Poste
model = glm::translate(matrizSombrilla, glm::vec3(0.0f, 1.4f, 0.0f));
model = glm::scale(model, glm::vec3(0.04f, 1.4f, 0.04f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// --- 2. TECHO Y PESTAÑAS---
glBindTexture(GL_TEXTURE_2D, texRojo);

// PARTE SUPERIOR ¿
model = glm::translate(matrizSombrilla, glm::vec3(0.0f, 2.5f, 0.0f));
model = glm::scale(model, glm::vec3(1.5f, 0.02f, 1.5f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);
```

```
// PESTAÑA FRONTAL
model = glm::translate(matrizSombrilla, glm::vec3(0.0f, 2.3f, 1.5f));
model = glm::scale(model, glm::vec3(1.5f, 0.2f, 0.02f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// PESTAÑA TRASERA
model = glm::translate(matrizSombrilla, glm::vec3(0.0f, 2.3f, -1.5f));
model = glm::scale(model, glm::vec3(1.5f, 0.2f, 0.02f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// PESTAÑA DERECHA
model = glm::translate(matrizSombrilla, glm::vec3(1.5f, 2.3f, 0.0f));
model = glm::scale(model, glm::vec3(0.02f, 0.2f, 1.5f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// PESTAÑA IZQUIERDA
model = glm::translate(matrizSombrilla, glm::vec3(-1.5f, 2.3f, 0.0f));
model = glm::scale(model, glm::vec3(0.02f, 0.2f, 1.5f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

// =====
// MÁSTIL DE LA BANDERA Y BANDERA
// =====
// Mástil
glBindVertexArray(VAO); // VAO de los cubos normales
glBindTexture(GL_TEXTURE_2D, texGris); //

model = matrizSombrilla;
// Lo posicionamos en el centro y lo subimos
model = glm::translate(model, glm::vec3(0.0f, 3.1f, 0.0f));
model = glm::scale(model, glm::vec3(0.02f, 0.6f, 0.02f));
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
glDrawArrays(GL_TRIANGLES, 0, 36);

float tiempo = glfwGetTime();
float velocidad = 3.8f;
float amplitud = 10.0f; // Curvatura de la tela
float desfase = 0.65f; // Retraso entre segmentos para el efecto de onda

glBindVertexArray(flagVAO); // Cambiamos al VAO de los triángulos
glBindTexture(GL_TEXTURE_2D, texNaranja);

//matriz base para la bandera, anclada al tope del nuevo mástil
glm::mat4 matrizOnde = matrizSombrilla;
matrizOnde = glm::translate(matrizOnde, glm::vec3(0.0f, 3.6f, 0.0f));

// 4 segmentos para el ondeo
for (int i = 0; i < 4; i++) {
    float angulo = sin(tiempo * velocidad - (i * desfase)) * amplitud;

    // Rotación principal: Ondeo horizontal (Eje Y)
    matrizOnde = glm::rotate(matrizOnde, glm::radians(angulo), glm::vec3(0, 1, 0));

    // Rotación secundaria: Eje Z para realismo
    float aleteo = cos(tiempo * 4.5f - i) * 3.0f;
```

```
        matrizOnde = glm::rotate(matrizOnde, glm::radians(aleteo), glm::vec3(0, 0, 1));
        model = matrizOnde;
        glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));

        // Dibujamos el segmento correspondiente (cada uno tiene 6 vértices)
        glDrawArrays(GL_TRIANGLES, i * 6, 6);
        // Nos movemos al final de la pieza actual (0.2f) para que la siguiente nazca ahí
        matrizOnde = glm::translate(matrizOnde, glm::vec3(0.2f, 0.0f, 0.0f));
    }

    glBindVertexArray(0);

    glfwSwapBuffers(window);
}

// Terminate GLFW, clearing any resources allocated by GLFW.
glfwTerminate();

return 0;
}

void Animation() {
    if (animBotella) {
        rotBotella += 100.0f * deltaTime;
    }

    if (AnimBall) {
        // --- Velocidad de las patas---
        // Multiplicamos por deltaTime para que sea suave
        float velocidadPatas = 100.0f * deltaTime;

        if (!step) {
            p1 += velocidadPatas;
            p2 -= velocidadPatas;
            if (p1 > 25.0f) step = true;
        }
        else {
            p1 -= velocidadPatas;
            p2 += velocidadPatas;
            if (p1 < -25.0f) step = false;
        }

        // --- Velocidad de (Desplazamiento) ---
        pagPos.x -= 1.3f * deltaTime;
    }

    if (luzAnim) {
        if (pointLightPositions[0].y > 0.0f) {
            ml -= 10.0f * deltaTime;
            pointLightPositions[0].y = 6.45f + ml;
        }

        // 2. DETECCIÓN DE PISO: Si ya llegó o se pasó del suelo
        if (pointLightPositions[0].y <= 0.0f) {
```

```
        pointLightPositions[0].y = 0.0f;

        // --- APAGAR LUZ ---
        Light1 = glm::vec3(0.0f);
        active = false;
        luzAnim = false;    // Detenemos la animación de caída
    }
}

// Moves/alters the camera positions based on user input
void DoMovement()
{
    // Camera controls
    if (keys[GLFW_KEY_W] || keys[GLFW_KEY_UP])
    {
        camera.ProcessKeyboard(FORWARD, deltaTime);
    }

    if (keys[GLFW_KEY_S] || keys[GLFW_KEY_DOWN])
    {
        camera.ProcessKeyboard(BACKWARD, deltaTime);
    }

    if (keys[GLFW_KEY_A] || keys[GLFW_KEY_LEFT])
    {
        camera.ProcessKeyboard(LEFT, deltaTime);
    }

    if (keys[GLFW_KEY_D] || keys[GLFW_KEY_RIGHT])
    {
        camera.ProcessKeyboard(RIGHT, deltaTime);
    }

    if (keys[GLFW_KEY_T])
    {
        pointLightPositions[0].x += 0.01f;
    }
    if (keys[GLFW_KEY_G])
    {
        pointLightPositions[0].x -= 0.01f;
    }

    if (keys[GLFW_KEY_Y])
    {
        pointLightPositions[0].y += 0.01f;
    }

    if (keys[GLFW_KEY_H])
    {
        pointLightPositions[0].y -= 0.01f;
    }
}
```

```
        if (keys[GLFW_KEY_U])
        {
            pointLightPositions[0].z -= 0.1f;
        }
        if (keys[GLFW_KEY_J])
        {
            pointLightPositions[0].z += 0.01f;
        }
    }

// Is called whenever a key is pressed/released via GLFW
void KeyCallback(GLFWwindow* window, int key, int scancode, int action, int mode)
{
    if (GLFW_KEY_ESCAPE == key && GLFW_PRESS == action)
    {
        glfwSetWindowShouldClose(window, GL_TRUE);
    }

    if (key >= 0 && key < 1024)
    {
        if (action == GLFW_PRESS)
        {
            keys[key] = true;
        }
        else if (action == GLFW_RELEASE)
        {
            keys[key] = false;
        }
    }

    if (keys[GLFW_KEY_SPACE])
    {
        active = !active;
        if (active)
        {
            Light1 = glm::vec3(1.0f, 1.0f, 0.0f);
        }
        else
        {
            Light1 = glm::vec3(0); // Cuando es solo un valor en los 3 vectores pueden dejar solo
una componente
        }
    }

    // Controles de animación (N: Pájaro, P: Luz, O: Botella)
    if (keys[GLFW_KEY_N]) AnimBall = !AnimBall;
    if (keys[GLFW_KEY_P]) luzAnim = !luzAnim;
    if (keys[GLFW_KEY_O]) animBotella = !animBotella;
}

void MouseCallback(GLFWwindow* window, double xPos, double yPos)
{
    if (firstMouse)
    {
        lastX = xPos;
        lastY = yPos;
        firstMouse = false;
    }
}
```

```
GLfloat xOffset = xPos - lastX;  
GLfloat yOffset = lastY - yPos; // Reversed since y-coordinates go from bottom to left  
  
lastX = xPos;  
lastY = yPos;  
  
camera.ProcessMouseMovement(xOffset, yOffset);  
}
```

Conclusiones

La realización de este proyecto permitió consolidar los conocimientos adquiridos durante el curso de Computación Gráfica, integrando técnicas de modelado procedural, carga de activos externos, mapeo de texturas y sistemas de iluminación dinámica.

El desarrollo representó un desafío técnico significativo, principalmente debido a la naturaleza incremental de la metodología en cascada; cada nueva funcionalidad (como la implementación de múltiples *Shaders*) requería una reestructuración del pipeline gráfico para asegurar la compatibilidad entre los modelos importados de Blender y los generados directamente en OpenGL.

Un aspecto clave fue el aprendizaje sobre el modelado jerárquico. Lograr que las extremidades del ave se articularan en relación con el torso, o que el plato de la antena rotara sobre un brazo móvil, nos permitió comprender la importancia del orden en las multiplicaciones de matrices de transformación. Finalmente, la interactividad mediante eventos de teclado y el uso de funciones trigonométricas para animaciones orgánicas como el ondeo de la bandera reafirmaron que la computación gráfica es una intersección directa entre la programación lógica y la geometría analítica.

MANUAL DE OPERACIÓN Y CONTROL DEL ENTORNO VIRTUAL

Introducción al Sistema

El presente documento constituye el manual oficial de operación para el entorno virtual interactivo desarrollado bajo los estándares de la API OpenGL. Este sistema implementa un motor gráfico capaz de gestionar mallas poligonales, sistemas de iluminación basados en el modelo de Phong y transformaciones jerárquicas en tiempo real. La arquitectura del software ha sido diseñada para ofrecer una experiencia inmersiva mediante una navegación en primera persona y una serie de eventos dinámicos controlados por el usuario.

Protocolos de Navegación y Movimiento

La interacción con el entorno se rige por un sistema de cámara de libre desplazamiento. La traslación del observador dentro del espacio tridimensional se realiza mediante las teclas de dirección convencionales W, A, S y D, las cuales permiten el avance, retroceso y desplazamiento lateral respectivamente. La orientación de la mirada y el ángulo de visión se gestionan mediante el movimiento periférico del ratón. Para finalizar la sesión de manera segura y cerrar el contexto gráfico, se debe emplear la tecla ESC.

Configuración y Gestión de Iluminación

El sistema cuenta con una configuración lumínica dinámica que responde a comandos directos. El control maestro de las fuentes de luz ambientales y puntuales se opera a través de la Barra Espaciadora, funcionando como un conmutador de estado global. Adicionalmente, se ha integrado una fuente de luz focalizada vinculada de forma persistente a los vectores de dirección de la cámara.

Activación de Animaciones e Interacción Dinámica

El entorno posee una serie de eventos cinemáticos que pueden ser disparados por el operador. Al activar estos disparadores, el sistema procesa transformaciones jerárquicas complejas:

Cinemática del Pájaro (Tecla N): Ejecuta un ciclo de caminado que coordina la traslación del modelo con la oscilación rítmica de sus extremidades.

Rotación de Objetos (Tecla O): Inicia un giro concéntrico sobre el eje horizontal de la botella, manteniendo la integridad de sus componentes mediante matrices de rotación compuestas.

Evento de Impacto de lámpara (Tecla P): Acciona la caída acelerada de la luminaria. El sistema detecta el plano del suelo para detener el movimiento y desactivar el componente de emisión lumínica simultáneamente.

Asimismo, elementos como el sube y baja, la antena satelital y el ondeado de la bandera operan bajo rutinas de animación constantes, proporcionando una sensación de dinamismo continuo en el entorno exterior.